

**FACULTY
OF MATHEMATICS
AND PHYSICS**
Charles University

DOCTORAL THESIS (REPORT)

Mgr. Jáchym Bártík

A Unified Framework for Multi-Model Data Management and Adaptation

Department of Software Engineering

Supervisor of the doctoral thesis: doc. RNDr. Irena Holubová, Ph.D.

Advisors of the doctoral thesis: Ing. Pavel Koupil, Ph.D.

prof. RNDr. Roman Barták, Ph.D.

Study programme: Computer Science

Study branch: Software Systems

Prague 2026

The results of the thesis were achieved in the period of a doctoral study at the Faculty of Mathematics and Physics, Charles University in Prague in years 2022-2026.

Student: Mgr. Jáchym Bártík
Department of Software Engineering
Faculty of Mathematics and Physics
Charles University
Malostranské náměstí 25, 11800 Prague 1, Czechia

Supervisor: doc. RNDr. Irena Holubová, Ph.D.
Department of Software Engineering
Faculty of Mathematics and Physics
Charles University
Malostranské náměstí 25, 11800 Prague 1, Czechia

Advisors: Ing. Pavel Koupil, Ph.D.
Department of Software Engineering
Faculty of Mathematics and Physics
Charles University
Malostranské náměstí 25, 11800 Prague 1, Czechia

prof. RNDr. Roman Barták, Ph.D.
Department of Theoretical Computer Science and
Mathematical Logic
Faculty of Mathematics and Physics
Charles University
Malostranské náměstí 25, 11800 Prague 1, Czechia

Opponents: Dr. Eric Pardede
Department of Computer Science and Information
Technology
School of Computing, Engineering & Mathematical Sciences
La Trobe University
Melbourne Victoria 3086, Australia

prof. RNDr. Tomáš Pitner, Ph.D.
Department of Informatics
Faculty of Business and Economics
Mendel University in Brno
Zemědělská 1665/1, 613 00 Brno, Czech Republic

The thesis defense will take place on the 22nd of September 2025 at 16:00 in front of a committee for thesis defense in the branch P4I2 (Software Systems) at the Faculty of Mathematics and Physics, Charles University, Malostranské náměstí 25, 11800 Prague 1, Czechia

Chairman of Academic Council P4I2:

doc. RNDr. Jan Kofroň, Ph.D.

Department of Distributed and Dependable Systems

Faculty of Mathematics and Physics

Charles University

Malostranské náměstí 25, 11800 Prague 1, Czechia

The thesis can be viewed at the Study Department of Doctoral Studies of the Faculty of Mathematics and Physics, Charles University, Ke Karlovu 3, Prague 2. This abstract was distributed on the 8th of September 2026.

Title: A Unified Framework for Multi-Model Data Management and Adaptation

Author: Mgr. Jáchym Bártík

Department: Department of Software Engineering

Supervisor: doc. RNDr. Irena Holubová, Ph.D., Department of Software Engineering

Advisors: Ing. Pavel Koupil, Ph.D., Department of Software Engineering, and prof. RNDr. Roman Barták, Ph.D., Department of Theoretical Computer Science and Mathematical Logic

Abstract: Multi-model databases combine relational, document, graph, and other data models, but their management remains fragmented across incompatible schemas, query languages, and adaptation mechanisms. Building on an existing category-theoretic foundation for multi-model schemas, mappings, transformations, and querying, this thesis develops methods for propagating conceptual schema changes to logical schemas, and queries, as well as a self-adaptation approach that searches for better logical configurations under changing workloads. The adaptation process combines workload-driven search with learned cost estimation, allowing candidate configurations to be compared only from query plans. The thesis contributes formal models, propagation and synchronization algorithms, and an experimental validation. Together, the results show that multi-model data management can be described coherently, automated to a substantial degree, and adapted to evolving application needs.

Keywords: multi-model databases, query processing, schema evolution, self-adaptation, category theory

Contents

1	Introduction	1
2	Vision	2
2.1	Level of Automation	2
2.2	Challenges	3
2.3	Previous Work	3
2.4	Research Questions	3
3	Foundations	4
3.1	Common Terminology	4
3.2	Category Theory	5
3.3	Modeling, Transformations, and Querying	6
3.4	Related Work	7
4	Evolution Management	8
4.1	Evolution Hierarchy	8
4.1.1	Schema Modification Operations	9
4.1.2	Propagation to Data and Queries	10
4.2	Level of Automation	10
4.3	Contributions	11
5	Self-Adaptation	11
5.1	Problem Formulation	12
5.2	Search Strategy and Latency Estimation	13
5.3	Contributions	13
6	Complementary Approaches to Multi-Model Data Management	13
6.1	Benchmarking & Evaluation	14
6.2	Schema-free Query Unification	15
6.3	Contributions to Evaluation and Interoperability	16
7	Conclusion	16
	References	18
	List of Abbreviations	23
	List of Publications	24

1 Introduction

For many years, database research focused primarily on relational databases, which organize data into tables with fixed schemas [1]. The rise of **NoSQL** databases in the early 2000s popularized additional data models and formats, including document, graph, key-value, and wide-column data [2].

These models are well suited to different types of applications and workloads. They also introduce a new difficulty: moving data between systems that use different models is considerably more complex and error-prone than migrating data between relational databases. Moreover, the **NoSQL** ecosystem still lacks unified theoretical foundations, common standards, and tools for managing data across models. [3]

Example 1. The *Yelp Academic Dataset* contains JSON collections describing businesses, users, reviews, tips, friendships, check-ins, and photos. We use a subset of this dataset in which the **user** data is represented as a relational table, **business** as a document collection, **FRIEND** as a graph relationship, and **review** as a wide-column family. The representations remain connected through shared identifiers, which allows results from different models to be combined. The resulting multi-model representation is shown in Figure 1. $\square$

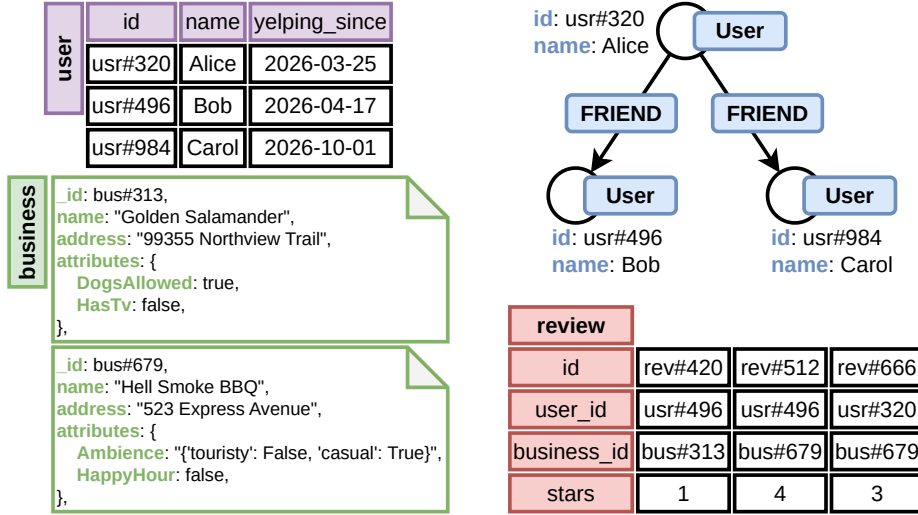


Figure 1 Multi-model representation from Example 1: a subset of the Yelp Academic Dataset represented in multiple data models and connected through shared identifiers.

Each model offers a different advantage. Relational data is suitable for structured data with fixed schemas, documents naturally represent semi-structured and nested data, graphs support queries over connectivity, and wide-column stores are optimized for high-throughput workloads. The same specialization makes cross-model use difficult: combining results from several databases is not straightforward and can introduce semantic inconsistencies if done incorrectly.

This setting is commonly called **multi-model data**. Using a separate database system for each model is known as **polyglot persistence** [2]. **Polystores** provide a unified interface over several databases [4], while **multi-model databases** combine several models within one system [3]. Both approaches still have to address the heterogeneity of the underlying models and systems; moreover, no multi-model database currently supports all commonly used data models [5].

These challenges motivated us to develop a unified approach to multi-model data management, presented on the following pages.

This report follows the same structure as the thesis itself:

- The *Vision* section formulates the research goals and the challenges of multi-model data management.
- The *Foundations* section introduces the common terminology and summarizes our previous work on modeling, transformations, inference, and querying.
- *Evolution Management* describes the propagation of schema changes to logical schemas, data, and queries.
- *Self-Adaptation* considers workload-aware optimization over alternative multi-model configurations.
- *Complementary Approaches* presents benchmark generation and schema-free query unification, both of which provide additional support for the main research direction.

2 Vision

When it comes to data, our primary goal is to extract information from it by asking questions and obtaining answers, in a process commonly known as **querying**. For multi-model data, querying crosses the boundaries of different models and, in the case of polyglot persistence, different database systems as well.

Querying requires several supporting capabilities. We need to **model** data so that we can reason about its structure and semantics, and in some cases we need to **infer** the schema from existing data. Modeling and inference are closely related to **evolution**: when a schema changes, we must propagate the change to the data and to the queries that depend on it. In a multi-model setting, this may also require us to **transform** data between different models.

Although our main concern is correctness, it is not the only one. The system should also query data efficiently and should be able to select an appropriate combination of models for a given workload. Because doing so manually is difficult, we want the system to adapt its internal representation through **self-adaptation**.

This vision, introduced in Paper I, combines modeling, inference, transformation, querying, evolution, and optimization within one multi-model data management framework [6, 7].

2.1 Level of Automation

We introduce automation gradually, distinguishing three levels according to the primary actor, that is, the main source of decisions and changes [6]:

- **User-driven** – users explicitly *model* the data, create *queries*, *transform* the data, and *evolve* the schema; the system propagates the changes to the data and queries.

- **Data-driven** – the system *infers* the schema from the data and updates it when the structure of the data changes.
- **System-driven** – the system uses *self-adaptation* to analyze the workload and data characteristics and proactively *transforms* the data into a configuration suitable for the current workload.

These levels are not strictly disjoint; they describe the dominant source of decisions and control. More automation reduces manual work and can reveal solutions that would be difficult to find manually, but it also increases system complexity and may reduce the user’s control over the process.

2.2 Challenges

The heterogeneity of the participating models is the main source of complexity [7]. Document, wide-column, and key-value models are often aggregate-oriented, while relational and graph models tend to decompose data into records connected by references or relationships. Some models are schema-less or only partially schema-full, whereas relational models generally require a fixed schema. These differences also affect query languages, integrity assumptions, and optimization techniques. There is consequently no one-size-fits-all solution: the best representation depends on the data, the workload, and the capabilities of the underlying systems.

Classical conceptual modeling approaches such as [ER](#) and [UML](#) provide useful technology-independent descriptions of a domain, but they do not by themselves provide mappings, transformations, and query execution across heterogeneous models [8, 9]. Similarly, polystore and multistore systems combine existing database technologies but must still manage the semantic relationships between their representations. Our vision treats conceptual modeling as the semantic basis for these broader operations.

2.3 Previous Work

The research presented in this thesis is based on a previous line of work on modeling [10, 11], querying [12, 13], and inference [14, 15] of multi-model data. In the following sections, we will use “we” and “our” to reference the previous work of our research group on these topics.

Other topics, i.e., evolution, self-adaptation, benchmarking, and schema-free query unification, are novel contributions of this thesis.

2.4 Research Questions

We formulate the vision as concrete research questions. Our previous work already addressed some of them:

- **Modeling** – How can heterogeneous data models be represented in a common semantic framework without erasing their model-specific structure?
- **Transformations** – How can data be transformed between different models without the need to write custom transformation rules for each pair of models?

- **Querying** – How can queries be unified across different data models, supporting a common, practically useful subset of features of each model, and allowing for cross-model queries and optimizations?

We use these answers as a foundation for the rest of our research. In this thesis, we address the following questions:

- **Evolution** – How can changes be propagated across conceptual and logical schemas, data, and queries in a multi-model environment? How can uncertain or ambiguous evolution decisions be supported by data-derived semantic evidence?
- **Self-adaptation** – How can a multi-model system choose suitable representations for a workload without exhaustively materializing and benchmarking all alternatives?
- **Benchmarking** – How can such systems be evaluated when real multi-model benchmarks and workloads are scarce?
- **Schema-free Query Unification** – How can multi-model queries be unified when a shared conceptual schema is unavailable?

3 Foundations

The preexisting work, mentioned in Section 2.3, is based on category theory. It provides a formal framework for modeling, transformations, inference, and querying of multi-model data, allowing us to unify these operations for different data models and database systems.

3.1 Common Terminology

We distinguish three abstraction levels commonly used in database design: **conceptual**, **logical**, and **physical**. A conceptual schema is a *platform-independent* description of the domain. It captures concepts and their relationships without committing to a particular database technology. A logical schema is *platform-specific*—it describes how selected parts of the conceptual schema are represented in a concrete data model. Finally, a physical schema describes the data structures used to store the data and implement indexes. We focus primarily on the conceptual and logical levels, but consider physical aspects when they affect the overall approach.

This separation allows us to map one conceptual schema to several logical representations. The mappings preserve the semantics of the domain while allowing each database to use the structures and operations native to its model.

Each data model has its own terminology, so we define model-agnostic terms:

- At the logical level, a **kind** is a class of items, such as a relational table, a document collection, or a collection of nodes or edges with the same label. An **attribute** is a field of a kind, such as a table column, document field, or node property.

- At the physical level, a **record** is an item of a kind, and a **value** is the value of an attribute in a record.
- At the conceptual level, a **simple type** is a value-carrying element (e.g., id, name), while a **complex type** is a structure consisting of simple types (e.g., user, business). A **projection** is a directed connection from a complex type to another simple or complex type (e.g., a user has a name). The **properties** [of a complex type] are all the simple types connected to it by a projection.

These terms abstract away model-specific details while preserving the structure needed for querying, transformation, and evolution.

3.2 Category Theory

Category theory provides the formal language for describing the structures and relationships in our approach. A **small category** $\mathcal{C} = (\mathcal{O}, \mathcal{M}, \circ)$ consists of a set of **objects** $\mathcal{O}$, a set of **morphisms** $\mathcal{M}$, and a composition operation $\circ$ over morphisms. A morphism $f \in \mathcal{M}$ is a directed connection between two objects $A, B \in \mathcal{O}$, denoted $f : A \rightarrow B$. For each object $A \in \mathcal{O}$, there is an **identity morphism** $1_A : A \rightarrow A$. Finally, the composition operation $\circ$ is associative and has the identity morphisms as its neutral elements.

Another important concept in category theory is a **functor** which maps objects and morphisms between categories while preserving identities and composition.

In our setting, a conceptual schema is a small category called a **schema category**. Its objects represent simple and complex types, and its morphisms represent projections between them. The category therefore captures both the domain concepts and the paths through which they are related. A logical schema is also a schema category, but its objects and morphisms have the specific semantics of the underlying data model.

The schema category introduced in Example 2 is used throughout the report. The ovals represent conceptual objects, while the arrows represent projections. This category is not merely a diagram of the domain: it provides the structure against which mappings, query patterns, and schema modifications can be defined.

Example 2. The schema category for the multi-model representation from Example 1, depicted in Figure 2, contains objects for types such as **User**, **Review**, and **Business**, as well as their simple properties. Its morphisms represent projections such as the connection from **Review** to its identifier and the relationship between a review and a business. Each morphism is identified by a numeric **signature**. $\square$

Each logical schema must conform to the constraints of its data model. A relational schema is flat, whereas a document schema can express nested structures. The logical schema is connected to the conceptual schema by a **mapping**, which is a functor from the logical category to the conceptual category. This mapping preserves the semantics of the conceptual schema while allowing the logical representation to use model-specific structures, tailoring each logical schema to the capabilities of its underlying model.

Example 3. The mapping in Figure 3 illustrates how the conceptual structure can be realized differently. The wide-column model cannot store the complex **User** type as

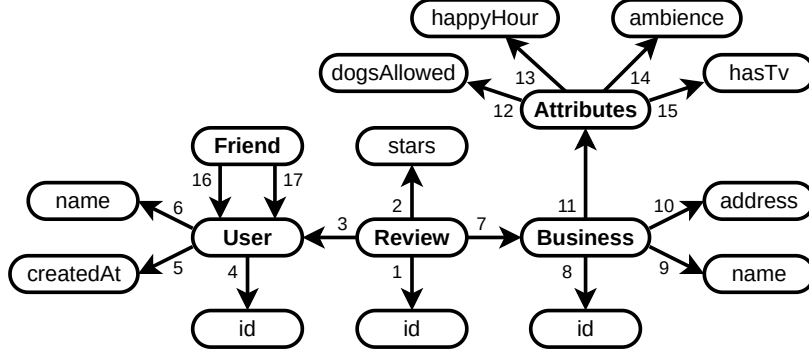


Figure 2 Schema category from Example 2, modeling the data from Figure 1. The objects represent conceptual types and the morphisms represent projections between them.

a nested column, but it can embed the simple type `id` into `review` as `user_id`. The corresponding composite morphism 3.4 represents the path through `Review` and `User` via morphisms 3 and 4. $\square$

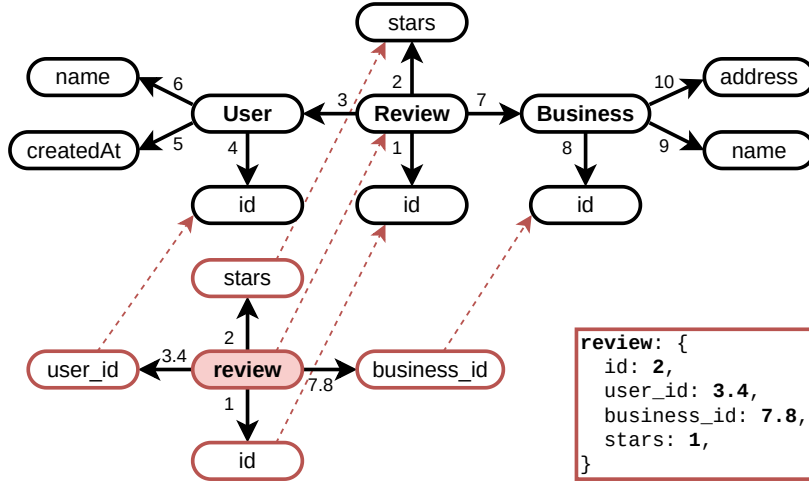


Figure 3 Mapping from Example 3 between a wide-column logical schema and the conceptual schema from Figure 2. The logical representation adapts the conceptual structure to the capabilities of the underlying model.

3.3 Modeling, Transformations, and Querying

Our previous work introduced an **instance category** with the same structure as the schema category and algorithms for converting data between databases and instance categories [10, 11]. This common representation allows us to transform data between supported models without defining a separate transformation for every pair of models. Database-specific wrappers provide the concrete data structures and the required **DDL** and **DML** statements, while the transformation algorithms remain model-independent [11].

Querying heterogeneous data is difficult because each database system has its own language, and because systems using the same model may still differ in syntax and semantics [7]. Existing extensions such as **SQL/JSON** [16] and **SQL/PGQ** [17]

cover particular combinations of models, but do not provide a general solution for cross-model queries. In a polyglot persistence system, users would otherwise have to write several native queries and combine their results manually.

These requirements led us to develop the **multi-model query language** (**MMQL**) [12, 13]. **MMQL** is defined over the schema category and is therefore independent of the underlying models and database systems. During execution, the query engine matches query patterns against the available logical schemas, splits a query into maximal subqueries executable by individual databases, translates them into native languages, executes them, and combines the results.

Example 4. Consider the query: *Find all reviews with at least 3 stars about a business with id bus#679, ordered by the number of stars (ascending). For each review, return the stars and the author’s name.* No single logical schema contains all required information, so **MMQL** splits the query into subqueries over the wide-column and relational databases and joins their results using the shared **User** identifier. $\square$

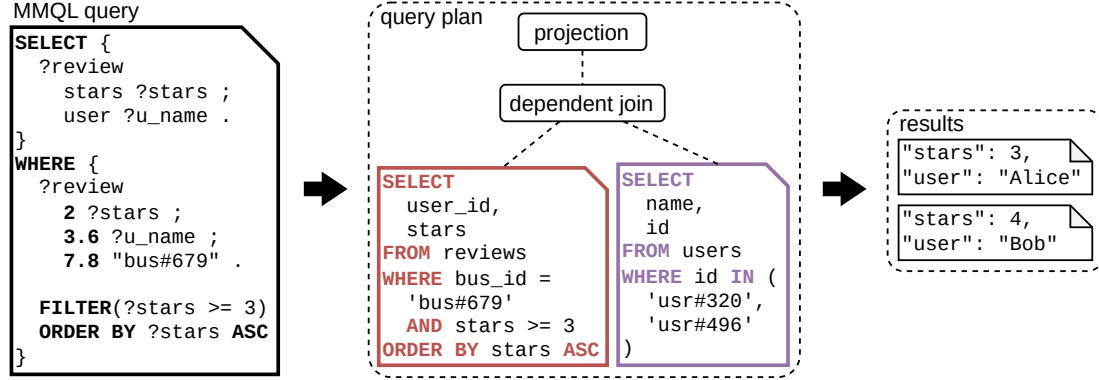


Figure 4 Multi-model query from Example 4. The query is split into subqueries executed in different databases and then combined according to the shared conceptual schema.

Because redundancy may make several query plans possible, the engine estimates their cost and selects the best one. We extended the query engine with predicate pushdown, dependent joins, and cache-based cost estimation [18] (under review). The same conceptual representation therefore supports not only a unified interface, but also cross-model query planning and optimization. These optimizations are contributions of this thesis, rather than part of the previous modeling and querying work.

3.4 Related Work

Traditional conceptual modeling approaches such as **ER** [8] and **UML** [9] were designed primarily for relational or relational-like data. They do not directly capture the structures of some **NoSQL** models or the relationships between several models and database systems. We use them as inspiration for the conceptual level, but we represent the conceptual schema as a category so that the same structure can also support mappings, transformations, evolution, and cross-model querying.

The closest practical line of work includes **U-schema** [19] and its extensions, such as **Orion** [20, 21] and **SkiQL** [22]. These approaches primarily unify data at the logical level. A single logical schema may fail to express the features

of all underlying models, while several disconnected logical schemas introduce redundancy and make their semantic relationships less explicit. Our approach retains one platform-independent conceptual schema and connects it to specialized logical schemas through mappings. This allows each database to use a suitable representation while preserving a shared semantic view. SkiQL also differs in its purpose: it queries schema structure, whereas [MMQL](#) queries data content over the conceptual schema.

Category theory has also been used to formalize heterogeneous data management and transformations [23, 24, 25]. These works provide rigorous categorical algebra and calculus for multi-model predicates, allowing us to model the multi-model data. Our approach has a broader scope—it uses category theory to support the full set of above-described operations. We also implement these operations over practical database systems to validate and evaluate the approach.

Finally, polystore and mediator-style systems provide unified access to heterogeneous backends [26, 27, 28]. Multi-model databases and standards such as [SQL/XML](#), [SQL/JSON](#), and [SQL/PGQ](#) extend individual systems or languages to additional models [7, 29, 16, 17]. These approaches demonstrate the value of interoperability, but generally treat local representations as given. Our framework combines the conceptual schema, logical representations, mappings, and queries in one foundation, which lets us reason about both interoperability and the choice of representation itself.

4 Evolution Management

Everything is subject to change, and multi-model data is no exception. Changes may occur in the conceptual or logical schema, in the data itself, or in the queries defined over the data. **Evolution management** involves detecting these changes, describing them formally, and propagating them to all affected components so that the system remains consistent and correct. This is especially challenging in a multi-model environment, where the relationships between components are complex and a local change can have consequences throughout the system.

The main challenges are to describe and propagate changes independently of the underlying data models, to provide an evolution language that captures both atomic operations and higher-level intent, and to propagate changes to data and queries. The system should also detect changes originating in the data, choose between alternative migration and synchronization strategies, and request user input only when the available information is insufficient for a reliable decision.

4.1 Evolution Hierarchy

Our approach starts with a hierarchy of components and rules for propagating changes between them. The conceptual schema is at the top of the hierarchy and can be modified through atomic *schema modification operations* (SMOs). These operations may be provided by the user or inferred from changes in the data. Changes to the conceptual schema are propagated to the logical schemas, and changes to the logical schemas are propagated to the data through [DDL](#) and [DML](#) statements.

Queries are defined over the conceptual schema and translated into native database queries only at execution time. Consequently, changes to logical schemas do not directly affect **MMQL** queries; queries are updated when the conceptual **SMOs** affect their selection or projection patterns.

Propagation can also proceed bottom-up: the system may observe a change in the data, infer the corresponding conceptual operation, and then apply the usual top-down propagation. Changes in the workload may trigger self-adaptation and lead to changes in logical schemas and data layouts. The resulting hierarchy is shown in Figure 5.

4.1.1 Schema Modification Operations

The **SMOs** were proposed in Paper II and formally defined in Paper IV. We divide them into three groups:

- **basic** operations work with individual objects and morphisms, such as **create**, **delete**, and **move**,
- **composite** operations capture frequent higher-level patterns, such as properties, relationships, arrays, sets, and maps (e.g., **createProperty**),
- **complex** operations capture larger restructurings, such as **group**, **join**, and **asMap**.

Users can therefore express changes at an appropriate level, while the system can reduce every operation to basic operations with explicit preconditions and postconditions. This makes propagation analyzable and allows the system to optimize, roll back, or postpone individual steps. We also distinguish between **light** changes, which update the conceptual schema and mappings but leave logical schemas and data unchanged, and **heavy** changes, which propagate to the logical schemas and data as well.

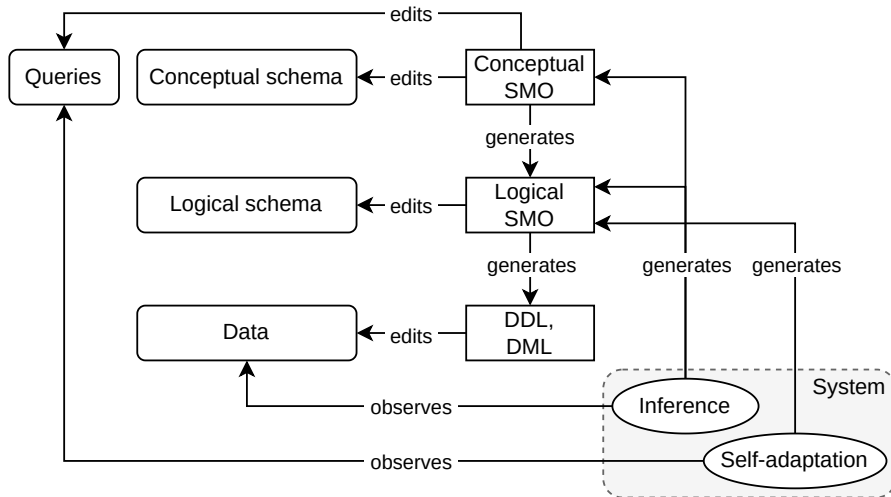


Figure 5 Hierarchy of components for evolution management. Changes are propagated downward from the conceptual schema, while changes originating in data or query workloads can be observed and propagated upward by inferring the corresponding operations.

4.1.2 Propagation to Data and Queries

The propagation to queries builds on the approach from Paper III. Each query is analyzed through its selection and projection categories, and the system evaluates how a basic SMO affects these categories and clauses such as **FILTER**, **DISTINCT**, **ORDER BY**, and aggregations. Some changes preserve the query behavior, some require a restricted rewrite, and some are too destructive to repair automatically.

The difficult cases are those in which the change decreases the available information, for example by deleting a property used by an existing query. The system may then reconstruct the missing values using a phantom representation, replace them using functional dependencies, or ask the user for additional guidance. **FDepHunter**, described in Paper V, provides semantic evidence for choosing between these alternatives by discovering and refining functional dependencies in real data.

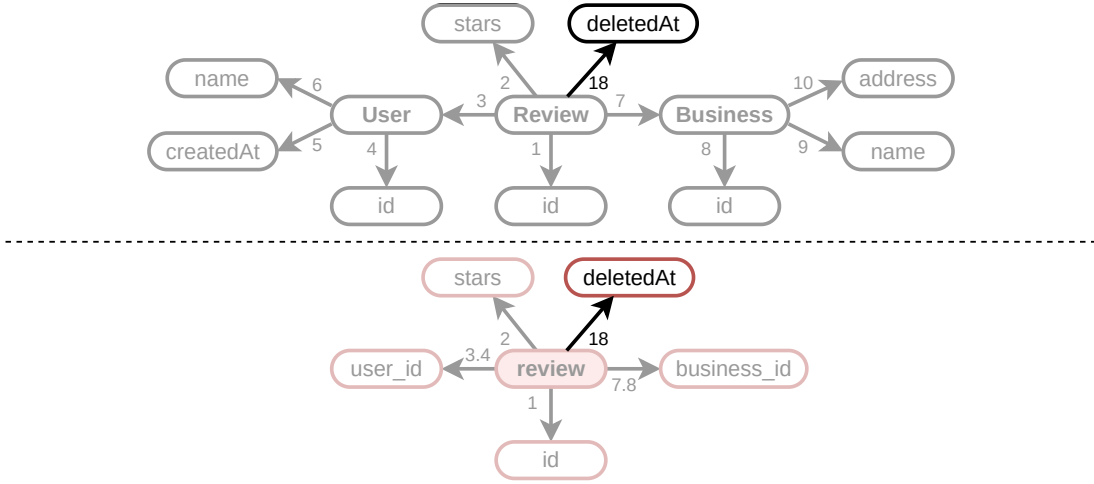


Figure 6 Propagation from Example 5: adding a property to the conceptual schema and propagating the change to a logical schema through schema modification operations.

Example 5. Starting with the conceptual schema and the wide-column logical schema, users can apply `createProperty` to add `deletedAt` to `Review`. As shown in Figure 6, the system updates the mapping and adds the corresponding attribute to the logical schema and data. Because the existing query does not use this property, it remains unchanged; users may nevertheless add the property later to filter out deleted reviews. More complex operations, such as converting the properties of `Attributes` into a key-value map, may require a query rewrite. $\square$

4.2 Level of Automation

We distinguish three levels of automation according to how much of the propagation process the system determines and executes [6]:

- **Rule-based adaptation** – the user or change-detection mechanism specifies what should change, and the system deterministically executes how it is propagated.
- **Learning-based adaptation** – the system chooses among several admissible propagation or synchronization strategies.

- **Self-adaptation** – the system decides that a change should be initiated based on workload observations and performance estimates.

Rule-based propagation is sufficient when the intended change and its consequences are clear. Learning-based adaptation becomes useful when several repairs are possible, for example when a deleted value can be reconstructed in several ways. FDepHunter supports this level by providing higher-quality information about functional dependencies through an interactive workflow in which domain experts can refine the discovered dependencies. Self-adaptation goes one step further and initiates structural changes in response to the workload; it is discussed in the following section.

4.3 Contributions

Evolution management is challenging even in a single-model setting. Relational approaches benefit from a mature theory, but generally assume one logical model and query language [30]. Work on **Darwin** supports lazy and hybrid migration strategies and version jumps for aggregate-oriented NoSQL stores [31], but does not provide a model-agnostic conceptual layer or query synchronization across a shared multi-model representation. The **Orion** language provides a useful taxonomy of schema modifications across several models [20, 21], but remains centered on logical unification. Query rewriting transforms queries into equivalent or suitable forms [32], while query adaptation reacts to changing execution conditions [33]; our problem is query synchronization after schema evolution. Existing synchronization work is mostly single-model [34], with partial extensions to NoSQL databases [35] and polystores [4]. Our contribution is a unified treatment of evolution as propagation across conceptual and logical schemas, mappings, data, and queries.

We defined atomic operations for modifying the conceptual and logical schemas and for propagating changes through the system. We also introduced composite and complex operations that express common restructuring intent while remaining reducible to the atomic core. This allows users to work with meaningful operations without sacrificing precise propagation, consistency checks, or rollback.

We extended the approach to query synchronization. After a schema change, the system determines whether a query remains valid, can be rewritten unambiguously, requires an approximation, or needs additional user input. We also implemented and evaluated MM-evocat and MM-evoque in Paper II and Paper III. Finally, FDepHunter strengthens the learning-based part of the approach by providing functional-dependency information for ambiguous synchronization decisions in Paper V.

5 Self-Adaptation

With the categorical representation of multi-model data and the capabilities for querying and evolution, we can focus on optimizing the data layout. The aim is to choose a suitable combination of models for a workload without requiring the user to enumerate and test all alternatives manually.

Before developing our search-based solution, we studied the related problem of index selection in heterogeneous systems in Paper VI. We compared indexing support in relational, graph, key-value, document, wide-column, multi-model, and multi-modal systems and assessed how existing index-selection methods transfer beyond relational databases.

The survey showed that this transfer is limited and uneven. Many methods assume capabilities that are common in relational systems but unavailable or weaker elsewhere, such as rich what-if interfaces, stable cost models, and a uniform space of supported indexes. It also showed that cost estimation is the main technical bottleneck. Most importantly, physical tuning assumes that the logical representation and the database placement are already fixed. In a multi-model environment, those choices may themselves dominate performance.

We therefore formulate self-adaptation as a search over alternative mappings, embeddings, and redundancies of the same conceptual schema across several databases. The physical-design space remains important, but the first step is to optimize the logical representation itself.

5.1 Problem Formulation

We assume a fixed conceptual schema, a workload consisting of weighted queries, and a set of available database applications. A **configuration** is a set of logical schemas that, if materialized with the data, allows the system to answer all queries in the workload. The goal is to minimize overall query latency while accounting for the cost of migrating to and maintaining the configuration (see Paper VII).

The search space is potentially unbounded and grows exponentially with the size of the conceptual schema. We make it finite by considering complex types as the basic building blocks. A configuration specifies in which database or databases each kind is stored and into which other kind, if any, it is embedded.

Example 6. The resulting state matrix, shown in Figure 7, has one column for each complex type and one row for each database. An entry indicates that a kind is not stored in the database (0), is stored without embedding (1), or is embedded into another kind (an identifier of the parent kind, e.g., B). The matrix can therefore represent both model assignment and cross-model redundancy while preserving a finite search space. $\square$

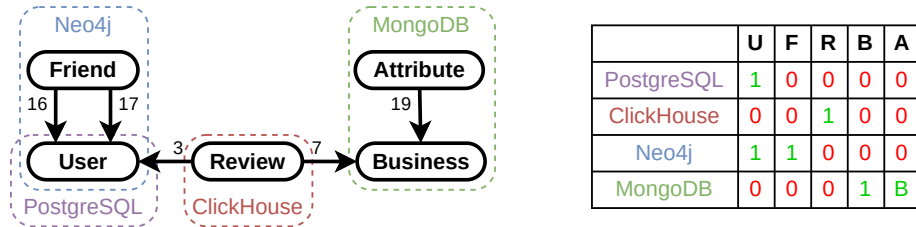


Figure 7 State representation from Example 6. Columns correspond to complex types and rows correspond to databases. The matrix captures model assignment, embedding, and cross-model redundancy.

5.2 Search Strategy and Latency Estimation

Even this finite state space is too large for exhaustive evaluation. We therefore explore it with [MCTS](#). The algorithm incrementally builds a directed acyclic graph of evaluated states. It selects a promising path while balancing exploration and exploitation, expands the selected state by applying a migration step, evaluates the resulting configuration, and propagates the value back through the graph. A graph is needed instead of a tree because the same configuration may be reached through different sequences of operations.

The main difficulty is evaluating a configuration without materializing it. We use learned latency estimators based on query-plan information. The same idea can be extended to cross-model queries by combining the estimated costs of their subqueries and intermediate results. We also estimate migration and maintenance costs by modeling the database operations required to reach and preserve a configuration. The estimates are necessarily approximate, but they make systematic search feasible.

5.3 Contributions

Existing physical-design methods such as index and materialized-view selection optimize structures within a fixed logical representation [36]. Learned tuning methods use workload behavior to predict latency or select system structures [37, 38, 39], but usually optimize an existing configuration rather than alternative mappings of one conceptual schema. Polystore and multistore systems address interoperation across backends [28, 27, 40], while migration approaches such as Darwin and MigCast focus on schema modification and migration [31, 41]. Our contribution is to make the logical representation itself part of the optimization problem.

We defined a finite state space that captures model assignment, embedding, and cross-model redundancy. The categorical representation makes it possible to describe queries independently of the configurations, define transitions through schema modification operations, and migrate data to a selected configuration. This connects the search process to the same semantic framework used for modeling, querying, and evolution.

On top of this representation, we introduced an automated search process based on weighted workloads and [MCTS](#). We implemented the approach in MM-mapsearch, including learned latency prediction, estimation of cross-model query costs, and an interactive interface for inspecting alternative configurations Paper VII. This establishes a path from reactive evolution to proactive adaptation: the system can use workload observations to identify a better representation and evaluate whether its expected benefit justifies migration and maintenance costs.

6 Complementary Approaches to Multi-Model Data Management

The following approaches complement the main framework. Benchmarking provides the datasets and workloads needed to evaluate modeling, querying,

evolution, and self-adaptation, while schema-free query unification addresses environments in which a shared conceptual schema is not yet available.

6.1 Benchmarking & Evaluation

Our modeling and optimization techniques require datasets and workloads that cover many kinds, alternative model combinations, complex cross-model queries, and multiple versions of a schema. Existing benchmarks are mostly single-model: TPC-H [42] and TPC-DS [43] target relational workloads, XMark [44] and DeepBench [45] target hierarchical or document data, and LDBC-SNB [46] targets graph workloads. Multi-model benchmarks such as BigBench [47], UniBench [48], and M2Bench [49] are important steps, but they generally provide only a small number of kinds and relatively simple queries, and usually describe one fixed schema and workload. They are therefore not sufficient for evaluating the large configuration spaces and evolution scenarios considered here.

Generating a synthetic dataset from scratch is time-consuming and often produces data that is not realistic enough. Transforming an existing dataset usually requires a new custom script for each target representation. Such scripts are difficult to reuse and validate semantically. Our approach, proposed in Paper VIII and extended in [50] (accepted after the thesis was submitted), uses the categorical framework to automate this process. Paper IX describes the implementation called **TransforMMer**.

The first step is to infer conceptual and logical schemas and the mappings between them [14, 15]. Because some semantic decisions cannot be recovered from the data alone, users then refine the inferred schema by confirming references, adding missing identifiers, or unifying structurally similar fragments. We restrict this refinement to light operations, which keeps the mappings consistent with the original logical schema.

Users can then select one or more target logical representations. TransforMMer transforms the data through the instance category and produces semantically equivalent relational, document, graph, wide-column, or other supported representations. The process and its alternative outputs are illustrated in Figure 8.

Example 7. Starting with the Yelp dataset from Example 1, TransforMMer infers a conceptual schema and mappings from the existing data. Users can confirm reference candidates and refine the schema, after which the system generates several semantically equivalent representations, such as two relational schemas with different degrees of embedding and a document schema with nested attributes. The resulting benchmark variants are shown in Figure 8. $\square$

The same framework supports benchmark evolution. Users can apply SMOs to create several versions of the conceptual schema; the system propagates the changes to the logical schemas and data so that all representations remain semantically aligned. Queries are defined in MMQL and evolved together with the schema. The query engine then generates comparable plans for the selected representations without executing them.

This changes the role of a benchmark. It is not merely a fixed dataset with a fixed workload, but a family of related data and workload variants that can expose how a system behaves under different representations and successive schema changes. Such a benchmark can evaluate querying, evolution, and self-adaptation

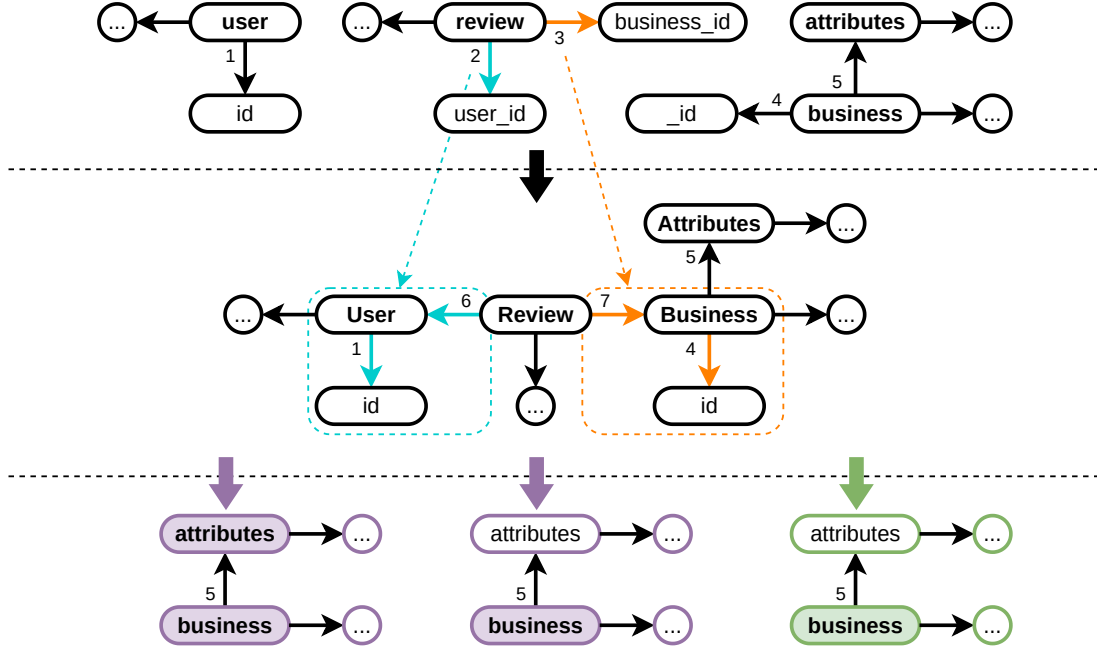


Figure 8 Benchmark creation from Example 7: inferred and refined conceptual schemas followed by several alternative logical representations.

within one controlled process.

6.2 Schema-free Query Unification

The categorical framework assumes an explicit conceptual schema and mappings. In highly dynamic settings such as data lakes, browser-side integration, or loosely coupled services, constructing this model before querying may be too costly or premature [3, 51]. In these cases, users still need to combine heterogeneous data and query languages. We address this problem in Paper X with **DortDB**.

Rather than introducing another universal user language, DortDB preserves established languages. **SQL** [52], **XQuery** [53], and **Cypher** [54] can be nested recursively into one another, without imposing a privileged top-level language. The mixed query is translated into a **unified algebra** covering relational, hierarchical, and graph-style operators [55, 56, 57]. This algebra acts as a common compilation target, so the system can inspect, optimize, and execute the mixed query as one plan.

Example 8 illustrates the approach. Different parts of a cross-model query may use different languages, and the system switches between them at any point using a language marker. It then derives a single algebraic plan instead of treating nested queries as opaque calls.

Example 8. This query expresses the same task as Example 4. Here, the outer query uses **Cypher** to match and filter reviews, while the nested **SQL** query retrieves the corresponding user.

```
MATCH (review:Review)
WHERE review.business_id = 'bus#679' AND review.stars >= 3
UNWIND (
  LANG sql
```

```

SELECT user.id, user.name
FROM user
WHERE user.id = review.user_id
) AS u
RETURN review.stars AS stars, u.name AS name
ORDER BY stars ASC

```

The **LANG** keyword switches languages inside the query. DortDB translates the complete query into the unified algebra, where the relational and graph-style operations can be inspected and optimized together. $\square$

DortDB is not a replacement for the categorical framework. The schema-first approach provides stronger semantic guarantees, explicit mappings, evolution machinery, and reasoning about alternative representations. DortDB starts with existing heterogeneous sources and focuses on rapid interoperability with minimal setup. Its prototype is intentionally in-memory and read-only, which allows us to focus on language interoperability and optimization while leaving storage and transaction management aside.

6.3 Contributions to Evaluation and Interoperability

TransforMMer contributes a transformation-driven approach to benchmark engineering. It derives families of semantically aligned multi-model datasets, logical representations, and workloads from existing data. Because the process records the conceptual changes and transformations, it supports reproducible evaluation across model combinations and evolution steps Papers VIII, IX. It also allows the same queries to be regenerated for each logical representation and schema version, making performance comparisons and evolution experiments directly comparable.

DortDB contributes a lightweight route to cross-model querying when an explicit schema is unavailable. Its unified algebra allows established query languages to interoperate symmetrically and provides a common basis for optimization (Paper X).

These two approaches address different prerequisites: TransforMMer supports systematic evaluation once semantic structure is available, whereas DortDB supports immediate querying when constructing that structure is not yet practical.

7 Conclusion

We address multi-model data management as one problem spanning conceptual modeling, schema inference, transformation, querying, evolution, and workload-driven adaptation. The central idea is to use a shared conceptual representation as a stable semantic layer, without erasing the differences between the logical models and database systems that realize it.

The main contributions of this thesis are as follows:

- *Evolution* We defined atomic, composite, and complex schema modification operations and showed how they can be propagated through conceptual schemas, logical schemas, data, and queries. We also analyzed how schema changes affect existing queries and distinguished among valid queries, safe

rewrites, approximate repairs, and cases that require additional user input. This allows the system to remain correct under schema evolution while using semantic evidence, including functional dependencies, to guide ambiguous decisions.

- *Self-adaptation.* We formulated workload-aware adaptation as a search over alternative logical configurations, where the objective is to choose a representation that matches the workload while accounting for migration and maintenance cost. The resulting approach uses a finite state space and [MCTS](#)-based optimization to identify better representations without exhaustively benchmarking every alternative.
- *Benchmarking.* We developed TransforMMer, which derives semantically aligned multi-model datasets, representations, and evolving workloads from existing data. This makes it possible to evaluate models, queries, and evolution scenarios under realistic and reproducible conditions.
- *Schema-free query unification.* We developed DortDB as a complementary route to heterogeneous querying when a shared conceptual schema is not yet available. By preserving established query languages and translating mixed queries into a shared algebra, the approach supports practical interoperability without requiring a full semantic model upfront.

Overall, the thesis demonstrates that multi-model data management should be treated as one integrated problem rather than as a collection of isolated techniques. The conceptual schema remains the semantic anchor, schema evolution and query synchronization protect correctness, self-adaptation improves the representation when the workload changes, and schema-free interoperability extends the approach to dynamic settings. Together, these contributions provide a coherent framework for modeling, querying, evolving, and adapting multi-model data in practice.

References

- [1] Edgar F. Codd. “Further Normalization of the Data Base Relational Model”. In: *IBM Research Report, San Jose, California* RJ909 (1971).
- [2] Pramod J. Sadalage and Martin Fowler. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. 1st. Addison-Wesley Professional, 2012. ISBN: 0321826620.
- [3] Jiaheng Lu and Irena Holubová. “Multi-model Databases: A New Journey to Handle the Variety of Data”. In: *ACM Comput. Surv.* 52.3 (June 2019). ISSN: 0360-0300. DOI: [10.1145/3323214](https://doi.org/10.1145/3323214).
- [4] Jérôme Fink, Maxime Gobert, and Anthony Cleve. “Adapting Queries to Database Schema Changes in Hybrid Polystores”. In: *Proceedings of SCAM ’20*. IEEE, 2020, pp. 127–131.
- [5] DB-Engines. *DB-Engines Ranking*. 2026. URL: <https://db-engines.com/en/ranking> (visited on June 25, 2026).
- [6] Irena Holubová, Pavel Koupil, and Jiaheng Lu. “Self-Adapting Design and Maintenance of Multi-Model Databases”. In: *Proceedings of the 26th International Database Engineered Applications Symposium. IDEAS ’22*. Budapest, Hungary: Association for Computing Machinery, 2022, pp. 9–15. ISBN: 9781450397094. DOI: [10.1145/3548785.3548810](https://doi.org/10.1145/3548785.3548810).
- [7] Qingsong Guo et al. “Multi-model query languages: taming the variety of big data”. English. In: *Distributed and Parallel Databases* (May 2023). ISSN: 0926-8782.
- [8] Peter P. Chen. “The Entity-Relationship Model—Toward a Unified View of Data”. In: *ACM Transactions on Database Systems* 1.1 (1976), pp. 9–36. DOI: [10.1145/320434.320440](https://doi.org/10.1145/320434.320440).
- [9] Object Management Group. *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. Formal specification. 2017.
- [10] Pavel Koupil, Martin Svoboda, and Irena Holubová. “MM-cat: A Tool for Modeling and Transformation of Multi-Model Data using Category Theory”. In: *Proceedings of MODELS ’21*. New York, NY, USA: IEEE, 2021, pp. 635–639.
- [11] Pavel Koupil and Irena Holubová. “A Unified Representation and Transformation of Multi-Model Data using Category Theory”. In: *Journal of Big Data* 9.1 (2022), p. 61.
- [12] Pavel Koupil, Daniel Crha, and Irena Holubová. “MM-quecat: A Tool for Unified Querying of Multi-Model Data”. In: *Proceedings of EDBT 2023*. OpenProceedings.org, 2023, pp. 831–834.
- [13] Pavel Koupil, Daniel Crha, and Irena Holubová. “A universal Approach for Simplified Redundancy-Aware Cross-Model Querying”. In: *Information Systems* 127 (2025). ISSN: 0306-4379.
- [14] Pavel Koupil, Sebastián Hricko, and Irena Holubová. “MM-infer: A Tool for Inference of Multi-Model Schemas”. In: *Proceedings of EDBT 2022*. OpenProceedings.org, 2022, 2:566–2:569.

- [15] Pavel Koupil, Sebastián Hricko, and Irena Holubová. “A universal approach for multi-model schema inference”. In: *J. Big Data* 9.1 (2022), p. 97. DOI: [10.1186/s40537-022-00645-9](https://doi.org/10.1186/s40537-022-00645-9).
- [16] ISO/IEC. *ISO/IEC TR 19075-6:2017 Information technology – Database languages – SQL – Part 6: SQL support for JavaScript Object Notation (JSON)*. 2017.
- [17] ISO/IEC. *ISO/IEC TR 9075-16:2023 Information technology – Database languages – SQL – Part 16: Property Graph Queries (SQL/PGQ)*. 2023.
- [18] Jáchym Bártík, Filip Štrobl, and Irena Holubová. *Reducing the Cross-Model Tax: Query Optimization over Multi-Model Data*. Manuscript under review. 2026. URL: <https://data.mmcatdb.com/papers/query-opt.pdf>.
- [19] Carlos Javier Fernández Candel, Diego Sevilla Ruiz, and Jesús Joaquín García Molina. “A Unified Metamodel for NoSQL and Relational Databases”. In: *Inf. Syst.* 104 (2022), p. 101898.
- [20] Alberto Hernández Chillón, Diego Sevilla Ruiz, and Jesús García Molina. “Towards a Taxonomy of Schema Changes for NoSQL Databases: The Orion Language”. In: *Proceedings of ER ’21, Virtual Event*. Vol. 13011. LNCS. Springer, 2021, pp. 176–185.
- [21] Alberto Hernández Chillón et al. “A Generic Schema Evolution Approach for NoSQL and Relational Databases”. In: *IEEE Trans. on Knowl. and Data Eng.* 36.7 (July 2024), pp. 2774–2789. ISSN: 1041-4347.
- [22] Carlos J. Fernández Candel, Jesús J. García-Molina, and Diego Sevilla Ruiz. “SkiQL: A unified schema query language”. In: *Data & Knowledge Engineering* 148 (Nov. 2023), p. 102234. ISSN: 0169-023X. DOI: [10.1016/j.datak.2023.102234](https://doi.org/10.1016/j.datak.2023.102234).
- [23] Valter Uotila and Jiaheng Lu. “A Formal Category Theoretical Framework for Multi-model Data Transformations”. In: *CoRR* abs/2201.04905 (2022). arXiv: [2201.04905](https://arxiv.org/abs/2201.04905).
- [24] Jiaheng Lu. *A Categorical Unification for Multi-Model Data: Part I Categorical Model and Normal Forms*. 2025. arXiv: [2502.19131](https://arxiv.org/abs/2502.19131) [cs.DB]. URL: <https://arxiv.org/abs/2502.19131>.
- [25] Jiaheng Lu. *A Categorical Unification for Multi-Model Data: Part II Categorical Algebra and Calculus*. 2025. arXiv: [2504.09515](https://arxiv.org/abs/2504.09515) [cs.DB]. URL: <https://arxiv.org/abs/2504.09515>.
- [26] Jiaheng Lu, Irena Holubová, and Bogdan Cautis. “Multi-model Databases and Tightly Integrated Polystores: Current Practices, Comparisons, and Open Challenges”. In: *Proceedings of CIKM 2018*. Torino, Italy: ACM, 2018, pp. 2301–2302.
- [27] Carlyna Bondiombouy and Patrick Valduriez. “Query processing in multi-store systems: an overview”. In: *Int. J. Cloud Comput.* 5.4 (2016), pp. 309–346.
- [28] Rana Alotaibi et al. “ESTOCADA: towards scalable polystore systems”. In: *Proceedings of the VLDB Endowment* 13.12 (2020), pp. 2949–2952.

- [29] ISO/IEC. *ISO/IEC 9075-14:2011 Information technology – Database languages – SQL – Part 14: XML-Related Specifications (SQL/XML)*. 2011.
- [30] David I Spivak and Ryan Wisnesky. “Relational Foundations for Functorial Data Migration”. In: *Proceedings of DBPL ’15*. New York, NY, USA: ACM, 2015, pp. 21–28. ISBN: 9781450339025.
- [31] Uta Störl and Meike Klettke. “Darwin: A Data Platform for NoSQL Schema Evolution Management and Data Migration”. In: *Proceedings of the Workshops of the EDBT/ICDT 2022 Joint Conference, Edinburgh, UK, March 29, 2022*. Vol. 3135. CEUR Workshop Proceedings. CEUR-WS.org, 2022.
- [32] Jan Kossmann, Thorsten Papenbrock, and Felix Naumann. “Data Dependencies for Query Optimization: A Survey”. In: 31.1 (June 2021), pp. 1–22. ISSN: 1066-8888.
- [33] Amol Deshpande, Zachary Ives, and Vijayshankar Raman. “Adaptive Query Processing”. In: *Found. Trends Databases* 1.1 (Jan. 2007), pp. 1–140. ISSN: 1931-7883.
- [34] Carlo A. Curino et al. “Update Rewriting and Integrity Constraint Maintenance in a Schema Evolution Support System: PRISM++”. In: *Proc. VLDB Endow.* 4.2 (Nov. 2010), pp. 117–128. ISSN: 2150-8097.
- [35] Mark Lukas Möller et al. “Query Rewriting for Continuously Evolving NoSQL Databases”. In: *Proc. ER 2019*. Vol. 11788. LNCS. 2019, pp. 213–221.
- [36] Stratos Idreos et al. “The Data Calculator: Data Structure Design and Cost Synthesis from First Principles and Learned Cost Models”. In: *Proceedings of SIGMOD ’18*. New York, NY, USA: ACM, 2018, pp. 535–550. ISBN: 9781450347037.
- [37] Ryan Marcus and Olga Papaemmanouil. “Plan-Structured Deep Neural Network Models for Query Performance Prediction”. In: *Proc. VLDB Endow.* 12.11 (July 2019), pp. 1733–1746. ISSN: 2150-8097.
- [38] Jun Sun et al. “Workload-Aware Performance Tuning for Multimodel Databases Based on Deep Reinforcement Learning”. In: *Int. J. Intell. Syst.* 2023 (Jan. 2023). ISSN: 0884-8173.
- [39] Feng Ye et al. “Parameters tuning of multi-model database based on deep reinforcement learning”. In: *J. Intell. Inf. Syst.* 61.1 (Nov. 2022), pp. 167–190. ISSN: 0925-9902.
- [40] Francesco Basciani et al. “TyphonML: A Modeling Environment to Develop Hybrid Polystores”. In: *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*. New York, NY, USA: Association for Computing Machinery, 2020, pp. 1–5. ISBN: 9781450381352.
- [41] Andrea Hillenbrand et al. “MigCast: Putting a Price Tag on Data Model Evolution in NoSQL Data Stores”. In: *Proceedings of SIGMOD ’19*. Amsterdam, Netherlands: ACM, 2019, pp. 1925–1928. ISBN: 9781450356435.
- [42] Transaction Processing Performance Council. *TPC Benchmark H (TPC-H) Standard Specification (version 3.0.1)*. 2022. URL: <https://www.tpc.org/tpch/> (visited on May 25, 2026).

- [43] Transaction Processing Performance Council. *TPC Benchmark DS (TPC-DS) Standard Specification (version 4.0.0)*. 2024. URL: <https://www.tpc.org/tpcds/> (visited on May 25, 2026).
- [44] Albrecht Schmidt et al. “XMark: a benchmark for XML data management”. In: *VLDB '02*. VLDB '02. Hong Kong, China: VLDB Endowment, 2002, pp. 974–985.
- [45] Stefano Belloni et al. “DeepBench: Benchmarking JSON Document Stores”. In: *Proceedings of the 2022 Workshop on 9th International Workshop of Testing Database Systems*. DBTest '22. Philadelphia, PA, USA: Association for Computing Machinery, 2022, pp. 1–9. ISBN: 9781450393539.
- [46] Orri Erling et al. “The LDBC Social Network Benchmark: Interactive Workload”. In: *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. SIGMOD '15. Melbourne, Victoria, Australia: Association for Computing Machinery, 2015, pp. 619–630. ISBN: 9781450327589.
- [47] Ahmad Ghazal et al. “BigBench: towards an industry standard benchmark for big data analytics”. In: *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*. SIGMOD '13. New York, New York, USA: Association for Computing Machinery, 2013, pp. 1197–1208. ISBN: 9781450320375.
- [48] Chao Zhang et al. “UniBench: A Benchmark for Multi-model Database Management Systems”. In: *TPCTC 2018*. Cham: Springer International Publishing, 2019, pp. 7–23. ISBN: 978-3-030-11404-6.
- [49] Bogyeong Kim et al. “M2Bench: A Database Benchmark for Multi-Model Analytic Workloads”. In: *Proc. VLDB Endow.* 16.4 (Dec. 2022), pp. 747–759. ISSN: 2150-8097.
- [50] Jáchym Bártík, Alžběta Šrůtková, and Irena Holubová. “Benchmark Engineering as a Design Instrument for Heterogeneous Information Systems”. In: *arXiv* (2026). Manuscript under second-round review. eprint: [2607.07175](https://arxiv.org/abs/2607.07175) (cs.DB). URL: <https://arxiv.org/abs/2607.07175>.
- [51] Amazon Web Services. “What is a Data Lake? - Introduction to Data Lakes and Analytics”. In: *AWS* (2023). URL: <https://aws.amazon.com/what-is/data-lake/>.
- [52] *ISO/IEC 9075-1:2023 Information Technology – Database Languages – SQL – Part 1: Framework (SQL/Framework)*. ISO/IEC, 2023.
- [53] Jonathan Robie, Michael Dyck, and Josh Spiegel. *XQuery 3.1: An XML Query Language*. <https://www.w3.org/TR/xquery-31/>. W3C Recommendation, 21 March 2017. Mar. 2017.
- [54] Nadime Francis et al. “Cypher: An Evolving Query Language for Property Graphs”. In: *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*. 2018, pp. 1433–1445. DOI: [10.1145/3183713.3190657](https://doi.org/10.1145/3183713.3190657).
- [55] Christopher Ré, Jérôme Siméon, and Mary F. Fernández. “A Complete and Efficient Algebraic Compiler for XQuery”. In: *Proceedings of ICDE 2006*. IEEE, 2006, p. 14. DOI: [10.1109/ICDE.2006.6](https://doi.org/10.1109/ICDE.2006.6).

- [56] Renzo Angles et al. “Path-based Algebraic Foundations of Graph Query Languages”. In: *Proceedings of EDBT 2025*. OpenProceedings.org, 2025, pp. 783–795. DOI: [10.48786/EDBT.2025.63](https://doi.org/10.48786/EDBT.2025.63).
- [57] Jürgen Hölsch, Michael Grossniklaus, and Marc H. Scholl. “Optimization of Nested Queries using the NF2 Algebra”. In: *Proceedings of SIGMOD 2016*. ACM, 2016. DOI: [10.1145/2882903.2915241](https://doi.org/10.1145/2882903.2915241).

List of Abbreviations

DDL Data Definition Language. [6](#), [8](#)

DML Data Manipulation Language. [6](#), [8](#)

ER Entity-Relationship. [3](#), [7](#)

MCTS Monte Carlo Tree Search. [13](#), [17](#)

MMQL Multi-Model Query Language. [7](#), [8](#), [9](#), [14](#)

NoSQL Not Only SQL. [1](#), [7](#), [11](#)

SMO Schema Modification Operation. [8](#), [9](#), [10](#), [14](#)

SQL Structured Query Language. [6](#), [8](#), [15](#)

UML Unified Modeling Language. [3](#), [7](#)

List of Publications

1. Pavel Koupil, Jáchym Bártík, and Irena Holubová. “MM-evocat: A Tool for Modelling and Evolution Management of Multi-Model Data”. In: *Proceedings of CIKM 2022*. Atlanta, GA, USA: ACM, 2022, pp. 4892–4896. ISBN: 9781450392365. DOI: [10.1145/3511808.3557180](https://doi.org/10.1145/3511808.3557180) (Paper II)
2. Jakub Klímek et al. “Atlas: A Toolset for Efficient Model-Driven Data Exchange in Data Spaces”. In: *Proceedings of MODELS 2023*. IEEE. 2023, pp. 4–8. DOI: [10.1109/MODELS-C59198.2023.00009](https://doi.org/10.1109/MODELS-C59198.2023.00009)
3. Pavel Koupil, Jáchym Bártík, and Irena Holubová. “MM-evoque: Query Synchronisation in Multi-Model Databases”. In: *Proceedings of EDBT 2024*. OpenProceedings.org, 2024, pp. 818–821. DOI: [10.48786/edbt.2024.78](https://doi.org/10.48786/edbt.2024.78) (Paper III)
4. Jáchym Bártík. “AI-Powered Orchestration of Multi-Model Data”. In: *Proceedings of VLDB 2024 PhD Workshop*. Guangzhou, China: VLDB Endowment, Aug. 2024. URL: https://vldb.org/2024/files/phd-workshop-papers/vldb_phd_workshop_paper_id_2.pdf (Paper I)
5. Jáchym Bártík, Pavel Koupil, and Irena Holubová. “Modelling and Evolution Management of Multi-Model Data”. In: *Proceedings of SAC 2024*. ACM, 2024
6. Jáchym Bártík, Alžběta Šrůtková, and Irena Holubová. “TransforMMer: A Universal Multi-Model Data Generator”. In: *Proceedings of EDBT 2025*. OpenProceedings.org, 2025, pp. 1150–1153. DOI: [10.48786/edbt.2025.111](https://doi.org/10.48786/edbt.2025.111) (Paper IX)
7. Pavel Koupil, Jáchym Bártík, and Irena Holubová. “Model-Agnostic Evolution Management”. In: *Proceedings of RCIS 2025*. Vol. 547. LNCS. Springer, 2025, pp. 245–261. DOI: [10.1007/978-3-031-92474-3_15](https://doi.org/10.1007/978-3-031-92474-3_15) (Paper IV)
8. Irena Holubová, Alžběta Šrůtková, and Jáchym Bártík. “Reshaping Reality: Creating Multi-Model Data and Queries from Real-World Inputs”. In: *Proceedings of ENASE 2025*. Jan. 2025, pp. 174–184. DOI: [10.5220/0013395300003928](https://doi.org/10.5220/0013395300003928) (Paper VIII)
9. Filip Ježek et al. “DortDB: Bridging Query Languages for Multi-Model Data Ponds”. In: *Proceedings of VLDB 2025* 18.12 (Aug. 2025), pp. 5335–5338. ISSN: 2150-8097. DOI: [10.14778/3750601.3750665](https://doi.org/10.14778/3750601.3750665) (Paper X)
10. Pavel Koupil et al. “FDepHunter: Harnessing Negative Examples to Expose Fakes and Reveal Ghosts”. In: *Proceedings of VLDB 2025* 18.12 (Aug. 2025), pp. 5227–5230. ISSN: 2150-8097. DOI: [10.14778/3750601.3750612](https://doi.org/10.14778/3750601.3750612) (Paper V)
11. Filip Mihál, Jáchym Bártík, and Pavel Koupil. “Refining Storage Strategy through Index Selection Methods in Multi-Model Database Systems: A

- Survey”. In: *Data & Knowledge Engineering* 164 (2026), p. 102582. ISSN: 0169-023X. DOI: [10.1016/j.datak.2026.102582](https://doi.org/10.1016/j.datak.2026.102582) (Paper VI)
12. Pavel Koupil et al. “MM-mapsearch: Workload-Aware Mapping Selection”. In: *Proceedings of EDBT 2026*. OpenProceedings.org, 2026. DOI: [10.48786/edbt.2026.68](https://doi.org/10.48786/edbt.2026.68) (Paper VII)
13. Jáchym Bártík, Alžběta Šrůtková, and Irena Holubová. “Benchmark Engineering as a Design Instrument for Heterogeneous Information Systems”. In: *arXiv* (2026). Manuscript under second-round review. eprint: [2607.07175](https://arxiv.org/abs/2607.07175) (cs.DB). URL: <https://arxiv.org/abs/2607.07175>
14. Filip Ježek et al. *One Algebra to Query Them All: A Schema-Agnostic Foundation for Cross-Model Querying*. Manuscript under review. 2026. URL: <https://data.mmcatdb.com/papers/unified-algebra.pdf>
15. Jáchym Bártík, Filip Štrobl, and Irena Holubová. *Reducing the Cross-Model Tax: Query Optimization over Multi-Model Data*. Manuscript under review. 2026. URL: <https://data.mmcatdb.com/papers/query-opt.pdf>